# DevOps Engineer

## Interview Preparation Handbook

Linux • Bash • Docker • GitHub Actions

*400+ Questions | 5+ Years Experience | India Market*

Basic → Advanced → Troubleshooting → Real-World Scenarios

| | | | |
|---|---|---|---|
| **Basic**<br>*Foundations* | **Intermediate**<br>*Core concepts* | **Advanced**<br>*Deep dives* | **Real-World**<br>*Scenarios* |

*Generated April 2026 — Prepared for Senior DevOps Engineer Interviews (India)*

# Linux

## BASIC — Foundations & File System

### 1. What is the Linux kernel and how does it differ from a distribution?

The kernel is the core of the OS that manages hardware, processes, memory, and I/O. A distribution (distro) bundles the kernel with utilities, package managers, and software (e.g., Ubuntu = kernel + apt + GNOME + tools).

### 2. Explain the Linux file system hierarchy (FHS).

/ is root. Key dirs: /bin (binaries), /etc (config), /var (variable data/logs), /tmp (temp), /home (users), /proc (virtual kernel state), /sys (hardware info), /dev (devices), /usr (user programs), /lib (libraries).

### 3. What is an inode?

An inode stores metadata about a file (owner, permissions, timestamps, pointers to data blocks) but NOT the filename. The directory maps names to inode numbers. Each file has exactly one inode.

### 4. Difference between soft link (symlink) and hard link?

Hard link: another directory entry pointing to the same inode — survives original deletion, same filesystem only. Symlink: pointer to a path — can span filesystems, breaks if target is deleted. Created with ln and ln -s.

### 5. What are file permissions in Linux and how do you read rwxr-xr--?

Three groups: owner/group/others. r=4 (read), w=2 (write), x=1 (execute). rwxr-xr-- = owner:rwx(7) group:r-x(5) others:r--(4) = octal 754.

### 6. How do chmod, chown, and chgrp differ?

chmod changes permissions, chown changes owner (and optionally group), chgrp changes only the group. Examples: chmod 755 file, chown user:group file, chgrp devops file.

### 7. What is the sticky bit and setuid/setgid?

Sticky bit on a directory (chmod +t) prevents users from deleting others' files (e.g., /tmp). Setuid (chmod u+s) runs executable as the file owner. Setgid on dir makes new files inherit the directory's group.

### 8. How do you find large files consuming disk space?

du -sh * | sort -rh | head -20 to find large dirs. find / -size +100M -type f to find files over 100MB. df -h for filesystem usage overview.

### 9. Explain /proc filesystem.

Virtual filesystem exposing kernel internals. /proc/cpuinfo (CPU info), /proc/meminfo (memory), /proc/<PID>/ (per-process details), /proc/net/ (network stats). Files don't persist on disk.

### 10. What is umask and how does it work?

umask subtracts permissions from the default (666 for files, 777 for dirs). umask 022 gives files 644, dirs 755. Check with umask, set with umask 027. Stored in /etc/profile or ~/.bashrc.

### 11. How do you view running processes?

ps aux (all processes), ps -ef, top (interactive), htop (enhanced), pgrep name (find PID by name), pidof name. For threads: ps -eLf.

### 12. Explain process states in Linux.

R (running), S (sleeping, interruptible), D (uninterruptible sleep, usually I/O), Z (zombie, exited but not reaped), T (stopped), I (idle kernel thread). Z processes indicate parent not calling wait().

### 13. What is the difference between kill, kill -9, and kill -15?

kill PID sends SIGTERM (15) — graceful shutdown request. kill -9 PID sends SIGKILL — immediate forced termination, cannot be caught. kill -15 is explicit SIGTERM. Always try SIGTERM before SIGKILL.

### 14. How does memory management work in Linux?

Physical RAM is divided into pages (4KB). Kernel uses virtual memory, swapping, and a page cache. Free memory isn't wasted — it's used for caching. Monitor with free -h, /proc/meminfo, vmstat.

### 15. What is swap space and when should you use it?

Swap is disk space used when RAM is full. It prevents OOM kills but is much slower than RAM. For production servers, having some swap (1-2x RAM or a fixed amount) as a safety net is recommended; swap usage in production typically indicates memory pressure.

### 16. How do you check disk I/O performance?

iostat -xz 1 (per-device stats), iotop (per-process I/O), vmstat 1 (system-wide), hdparm -tT /dev/sda (raw disk speed), blktrace for deep tracing.

### 17. Explain the boot process in Linux.

BIOS/UEFI → Bootloader (GRUB2) → Kernel loaded → initramfs (early userspace) → kernel mounts root fs → init/systemd (PID 1) → targets/runlevels → getty/login.

### 18. What is systemd and how does it differ from SysVinit?

systemd is a modern init system (PID 1) with parallel service startup, dependency management, socket/device activation, journald logging. SysVinit used sequential shell scripts. systemd uses unit files (.service, .target, .socket).

### 19. How do you manage services with systemctl?

systemctl start/stop/restart/reload service, systemctl enable/disable (boot), systemctl status service, systemctl list-units --type=service, journalctl -u service -f (logs).

### 20. What are runlevels/targets in systemd?

SysV runlevels map to systemd targets: 0=poweroff, 1=rescue, 3=multi-user.target, 5=graphical.target, 6=reboot. Set default with systemctl set-default multi-user.target.

### 21. How do you schedule tasks in Linux?

cron (crontab -e) for recurring tasks, at for one-time jobs, systemd timers (more flexible than cron, with logging). anacron for machines not always on.

### 22. Explain /etc/fstab.

Defines filesystems to mount at boot. Fields: device/UUID, mountpoint, filesystem type, options, dump flag, fsck order. Use UUID instead of /dev/sdX for stability. Edit carefully — bad entries prevent boot.

### 23. What are Linux namespaces?

Kernel feature isolating resources: PID, net, mnt, uts (hostname), ipc, user, cgroup namespaces. Basis for containers. Each namespace gives processes an isolated view of that resource type.

### 24. What are cgroups?

Control groups limit/account/isolate resource usage (CPU, memory, I/O, network) for process groups. v1 (legacy) and v2 (unified hierarchy, default in modern distros). Used by Docker, Kubernetes, systemd.

### 25. Explain Linux signals and common ones.

Signals are async notifications to processes. SIGTERM(15) graceful stop, SIGKILL(9) force kill, SIGHUP(1) reload config, SIGINT(2) Ctrl+C, SIGCHLD notify parent of child state change, SIGUSR1/2 application-defined.

## INTERMEDIATE — Networking, Users & Storage

### 26. How do you configure a static IP in Linux?

Ubuntu/Netplan: edit /etc/netplan/*.yaml with addresses, gateway4, nameservers, then netplan apply. RHEL: edit /etc/sysconfig/network-scripts/ifcfg-eth0 or use nmcli/nmtui. Verify with ip addr show.

### 27. Explain TCP/IP troubleshooting tools in Linux.

ping (ICMP reachability), traceroute/tracepath (path tracing), mtr (combined), ss -tulnp (socket stats, replaces netstat), ip route (routing table), dig/nslookup (DNS), curl -v (HTTP), tcpdump (packet capture), nmap (port scan).

### 28. What is iptables and how does it work?

iptables is a userspace tool for the netfilter kernel framework. Tables (filter, nat, mangle), chains (INPUT, OUTPUT, FORWARD, PREROUTING, POSTROUTING), rules matched top-down with a target (ACCEPT, DROP, REJECT, LOG). nftables is the modern successor.

### 29. How do you troubleshoot a server that's not reachable over SSH?

Check if sshd is running (systemctl status sshd), firewall rules (iptables -L, firewall-cmd --list-all), port listening (ss -tlnp | grep 22), /etc/hosts.deny, /var/log/auth.log for errors, SELinux/AppArmor denials (ausearch, dmesg), and cloud security groups.

### 30. What is LVM and how does it work?

Logical Volume Manager abstracts physical disks into PVs → VG → LVs. Enables dynamic resizing, snapshots, striping. Commands: pvcreate, vgcreate, lvcreate, lvextend + resize2fs/xfs_growfs. More flexible than raw partitions.

### 31. How do you extend an LVM partition online?

Add disk or expand PV: pvcreate /dev/sdb, vgextend vg0 /dev/sdb, lvextend -L +10G /dev/vg0/lv_data, then resize2fs /dev/vg0/lv_data (ext4) or xfs_growfs /mountpoint (XFS). No downtime needed.

### 32. Explain Linux user and group management.

useradd/adduser, usermod, userdel. Groups: groupadd, gpasswd -a user group. /etc/passwd (users), /etc/shadow (passwords), /etc/group. sudo access via /etc/sudoers (visudo). Lock accounts: passwd -l, usermod -L.

### 33. What is PAM and why is it important?

Pluggable Authentication Modules provide a flexible authentication framework. /etc/pam.d/ configs control login, ssh, su. Stacked modules (auth, account, password, session) allow MFA, LDAP integration, login restrictions, and password policies without modifying apps.

### 34. How do you configure SSH key-based authentication?

Generate key pair (ssh-keygen -t ed25519), copy public key to ~/.ssh/authorized_keys on target (ssh-copy-id or manual), set permissions (700 on ~/.ssh, 600 on authorized_keys). Disable password auth in /etc/ssh/sshd_config: PasswordAuthentication no.

### 35. Explain NFS and CIFS/SMB differences.

NFS (Network File System) is Linux-native, faster for Linux-to-Linux, uses RPC. CIFS/SMB is Windows-native (Samba implements it on Linux), better for mixed environments. NFS v4 adds security improvements. Mount with mount -t nfs or mount -t cifs.

### 36. What is RAID and common levels?

RAID 0 (striping, no redundancy), RAID 1 (mirroring, 50% usable), RAID 5 (striping + 1 parity, min 3 disks), RAID 6 (2 parity), RAID 10 (mirror+stripe). mdadm for software RAID on Linux. Use RAID for redundancy, not as a backup substitute.

### 37. How do you monitor system resources in real time?

top/htop (CPU/memory), iostat -x 1 (I/O), vmstat 1 (CPU/memory/swap), nethogs/iftop (network per process), dstat (combined), sar (historical via sysstat), Prometheus + node_exporter for production monitoring.

### 38. What is the difference between hard and soft limits (ulimit)?

Soft limits are the current effective limit (user can raise up to hard limit). Hard limits are the ceiling, only root can raise. ulimit -n (open files), -u (processes), -s (stack). Persistent in /etc/security/limits.conf. Critical for high-concurrency services.

### 39. Explain kernel parameters via sysctl.

sysctl reads/writes /proc/sys/* values at runtime. Persist in /etc/sysctl.conf or /etc/sysctl.d/*.conf, apply with sysctl -p. Examples: net.ipv4.ip_forward=1, vm.swappiness=10, net.core.somaxconn=65535, fs.file-max=2097152.

### 40. How does SELinux work and common commands?

SELinux enforces mandatory access control via security contexts (user:role:type:level). Modes: enforcing, permissive, disabled. Commands: getenforce, setenforce 0 (temporary permissive), sestatus, restorecon -Rv /path (fix contexts), audit2allow (create policies), ausearch -m avc (check denials).

### 41. What is AppArmor and how does it differ from SELinux?

AppArmor uses path-based profiles (easier to manage), while SELinux uses label-based type enforcement (more granular). AppArmor default on Ubuntu/Debian, SELinux default on RHEL/CentOS. aa-status, aa-enforce, aa-complain for AppArmor management.

### 42. How do you analyze log files efficiently?

journalctl -u service --since '1 hour ago', grep/awk/sed for text logs, tail -f /var/log/syslog, zgrep for compressed logs, logrotate for rotation, ELK/Loki/Splunk for aggregation. Key logs: /var/log/syslog, auth.log, kern.log, dmesg.

### 43. Explain the /etc/hosts file and DNS resolution order.

/etc/hosts is local name resolution. /etc/nsswitch.conf defines resolution order (typically: files dns). /etc/resolv.conf lists DNS servers. systemd-resolved manages DNS on modern systems. For containers, /etc/resolv.conf is often bind-mounted.

### 44. What are Linux capabilities?

Fine-grained privilege system breaking up root powers. Examples: CAP_NET_BIND_SERVICE (bind <1024 ports), CAP_SYS_ADMIN (broad admin), CAP_NET_RAW (raw sockets). View with getcap, set with setcap. Docker and Kubernetes use capabilities to reduce container privileges.

### 45. How do you trace system calls made by a process?

strace -p PID or strace command — shows all syscalls with arguments and return values. strace -e trace=network,file (filter by category). ltrace traces library calls. Useful for debugging permission issues, missing files, and performance bottlenecks.

### 46. Explain tmpfs and ramfs.

tmpfs is a virtual filesystem stored in RAM (and optionally swap), size-limited, contents lost on reboot. ramfs is similar but doesn't use swap and has no size limit (dangerous). /tmp, /run, /dev/shm are typically tmpfs. Use df -h to confirm.

### 47. What is D-Bus?

D-Bus is an IPC (inter-process communication) mechanism used by system services and desktop applications to communicate. systemd, NetworkManager, and many services use D-Bus. Useful to know when debugging why services aren't communicating.

### 48. How do you manage time synchronization in Linux?

NTP via chrony (preferred, chronyd) or ntpd. timedatectl status to check, timedatectl set-ntp true to enable. chronyc tracking/sources for status. Critical for log correlation, Kerberos, TLS certificates, and distributed systems.

### 49. What is dmesg and when do you use it?

dmesg shows kernel ring buffer — hardware events, driver messages, boot messages, kernel errors. dmesg -T (human timestamps), dmesg | grep -i error, watch dmesg for live monitoring. Essential for hardware issues, OOM kills, filesystem errors.

### 50. Explain Linux memory types: buff/cache vs free.

free -h shows: total RAM, used, free, shared, buff/cache, available. Buffer/cache is memory used for disk caching (reclaimable). Available is a better indicator of free memory than 'free'. Linux intentionally uses all RAM for caching to improve performance.

## ADVANCED — Kernel, Performance & Security

### 51. How does the Linux scheduler work (CFS)?

The Completely Fair Scheduler uses a red-black tree ordered by vruntime (virtual runtime). The task with least vruntime runs next. nice values adjust weight. Real-time tasks (SCHED_FIFO, SCHED_RR) have higher priority. cgroups limit CPU share per group.

### 52. Explain NUMA and its impact on performance.

Non-Uniform Memory Access: in multi-socket systems, memory access to local NUMA node is faster than remote. numactl --hardware to view topology, numastat to see hits/misses. Applications should be NUMA-aware. VMs inherit NUMA topology. Misaligned NUMA causes latency spikes.

### 53. What is huge pages and when to use them?

Normal pages are 4KB; huge pages are 2MB or 1GB. They reduce TLB misses for memory-intensive apps. Types: static (HugePages in /proc/meminfo), transparent (THP, automatic). Databases like Oracle require static huge pages. THP can cause latency spikes in some workloads.

### 54. How does the OOM killer work and how to tune it?

When memory is critically low, kernel kills processes to free memory. OOM score (/proc/PID/oom_score) determines victim. Tune with /proc/PID/oom_score_adj (-1000=never kill, 1000=kill first). vm.oom_kill_allocating_task, vm.panic_on_oom sysctls. Check via dmesg | grep -i oom.

### 55. Explain eBPF and its use cases.

Extended Berkeley Packet Filter runs sandboxed programs in the kernel without modules. Used for: observability (bpftrace, BCC tools), security (Falco, seccomp), networking (Cilium, XDP), performance profiling. Commands: bpftrace, bcc tools (execsnoop, opensnoop, tcplife).

### 56. What is io_uring and how does it improve I/O?

io_uring is a Linux async I/O interface using two ring buffers (submission/completion) shared between kernel and userspace. Reduces syscall overhead significantly vs epoll/select. Used by io-uring-based servers, RocksDB, and modern runtimes. Available since kernel 5.1.

### 57. How do you profile CPU performance on Linux?

perf stat (hardware counters), perf record + perf report (sampling profiler), perf top (live flamegraph input), flamegraph.pl for visualization, pprof, gprof. At the OS level: vmstat, mpstat -P ALL 1, sar -u. For call-graph: perf record -g.

### 58. Explain Linux network stack and packet path.

NIC → driver → ring buffer → softirq → netif_receive_skb → protocol stack (IP, TCP) → socket buffer → application. XDP (at driver level), tc (traffic control), netfilter hooks (iptables), conntrack all intercept at different points. Tune with ethtool, tc, sysctl net.*

### 59. What is DPDK and when would you use it?

Data Plane Development Kit bypasses the kernel network stack entirely, running drivers in userspace for line-rate packet processing. Used for high-performance NFV, firewalls, routers, telecom. Requires dedicated NICs/CPU cores. Not typical for most DevOps scenarios but relevant for infra-heavy roles.

### 60. How do you debug kernel panics?

Enable kdump (kernel crash dump): install kexec-tools, configure /etc/kdump.conf, enable kdump service. After panic, crash dumps in /var/crash. Analyze with crash utility + vmlinux. Also check dmesg, /var/log/kern.log, MCE hardware errors with mcelog.

### 61. Explain seccomp and how containers use it.

seccomp (secure computing mode) restricts syscalls a process can make. Modes: strict (only read/write/exit/sigreturn) and filter (BPF-based allowlist). Docker applies a default seccomp profile blocking ~44 syscalls. Kubernetes allows custom seccomp via securityContext. Critical for container hardening.

### 62. What is dirty page writeback and how to tune it?

When processes write to files, pages are marked dirty and periodically flushed to disk by pdflush/flush threads. Tune with vm.dirty_ratio (% RAM before sync write), vm.dirty_background_ratio (% before background flush), vm.dirty_expire_centisecs. High dirty data risks data loss on crash.

### 63. How does CPU pinning (affinity) work?

taskset -c 0,1 command pins a process to CPUs 0-1. cpuset cgroup restricts CPUs for a group. NUMA-aware pinning: numactl --cpunodebind=0. In Kubernetes: CPU Manager policy=static for Guaranteed QoS pods gets exclusive cores. Reduces cache thrashing and latency jitter.

### 64. Explain Linux socket internals and tuning for high concurrency.

Sockets use kernel buffers (send/recv). Key params: net.core.somaxconn (listen backlog), net.ipv4.tcp_max_syn_backlog, net.core.rmem_max/wmem_max (buffer sizes), net.ipv4.tcp_tw_reuse (TIME_WAIT reuse), net.ipv4.ip_local_port_range. epoll for O(1) event notification in high-concurrency servers.

### 65. What is kernel live patching?

Applying security patches to the running kernel without reboot. kpatch (Red Hat) and livepatch (Ubuntu/Canonical) tools. Works by redirecting function pointers to patched versions. Not all patches can be live-patched. Reduces downtime for security updates.

### 66. How do you analyze a core dump?

Enable core dumps: ulimit -c unlimited, set core_pattern (/proc/sys/kernel/core_pattern). Analyze with gdb executable corefile → bt (backtrace), info threads. For Java: jstack, jmap. systemd-coredump stores dumps in /var/lib/systemd/coredump, analyze with coredumpctl debug.

### 67. Explain TCP BBR congestion control.

BBR (Bottleneck Bandwidth and RTT) models network bandwidth and RTT to set sending rate, unlike CUBIC which reacts to packet loss. Better throughput on high-latency/lossy links. Enable: sysctl net.ipv4.tcp_congestion_control=bbr. Check: sysctl net.ipv4.tcp_available_congestion_control. Widely adopted in cloud environments.

### 68. What is auditd and how to use it for compliance?

auditd is the Linux audit daemon logging syscall events for security compliance (PCI-DSS, SOX, HIPAA). Rules in /etc/audit/rules.d/. Key commands: auditctl -w /etc/passwd -p wa -k passwd_changes (watch file), ausearch -k passwd_changes, aureport. Log to /var/log/audit/audit.log.

### 69. How does Linux handle time: monotonic vs wall clock?

Wall clock (CLOCK_REALTIME) can jump due to NTP corrections. Monotonic clock (CLOCK_MONOTONIC) only moves forward, used for measuring elapsed time. CLOCK_BOOTTIME includes sleep time. Critical for timeout calculations in applications. Use clock_gettime() with correct clock type.

### 70. Explain Linux memory zones (DMA, Normal, High).

x86-64: DMA zone (0-16MB, for legacy DMA devices), DMA32 (0-4GB), Normal (rest of RAM). Kernel allocates from zones based on hardware requirements. Zone balancing prevents zone exhaustion. ZONE_MOVABLE helps defragmentation. Mostly transparent but relevant for kernel module development.

### 71. What are kprobes and uprobes?

kprobes dynamically insert breakpoints into any kernel instruction for debugging/tracing. uprobes are similar but for userspace. Used by perf, bpftrace, SystemTap. Example: bpftrace -e kprobe:sys_execve to print exec path. Low overhead tracing without modifying code.

### 72. How do you harden a Linux server for production?

Minimal install, disable unnecessary services (systemctl disable), configure firewall (iptables/nftables/firewalld), SSH hardening (key auth only, disable root, change port), SELinux/AppArmor enforcing, fail2ban, auditd, unattended-upgrades, CIS benchmarks, Lynis audit, remove SUID binaries.

### 73. Explain IOMMU and its security benefits.

IOMMU (Input-Output Memory Management Unit) restricts DMA access by devices to specific memory regions. Prevents malicious/buggy devices from accessing arbitrary RAM (DMA attacks). Required for PCIe device passthrough in VMs (VFIO). Enable via intel_iommu=on or amd_iommu=on kernel parameter.

### 74. What is KSM (Kernel Same-page Merging)?

KSM scans memory for identical pages across processes and merges them into one copy-on-write page. Useful for VMs/containers running same OS images. Reduces memory usage significantly. ksmtuned daemon manages it. Check /sys/kernel/mm/ksm/. Trade-off: CPU overhead for scanning.

### 75. How do you debug network packet loss on Linux?

ip -s link show (interface errors/drops), ethtool -S eth0 (NIC counters), ss -s (socket stats), netstat -su (UDP drops), /proc/net/dev, tc -s qdisc (queueing disc stats), sar -n DEV, tcpdump/Wireshark for deep inspection, check irqbalance and NIC queue count vs CPU count.

## REAL-WORLD SCENARIOS & TROUBLESHOOTING

### 76. A production server has 100% CPU usage. Walk through your approach.

1. Identify: top -c/htop to find PID. 2. Inspect: ps -p PID -o pid,ppid,cmd,%cpu,%mem; pstree -p PID. 3. Profile: perf top -p PID or strace -p PID -c. 4. Check logs: journalctl -u service. 5. Runaway cron/OOM loop? zombie processes? 6. Kill if needed, capture core dump, analyze and fix root cause.

### 77. Disk is full on a production server. What's your step-by-step?

1. df -h (which FS). 2. du -sh /* (find large dirs), du -sh /var/log/* /tmp/*. 3. Large files: find / -xdev -size +500M. 4. Deleted but open files: lsof | grep deleted. 5. Quick fix: clear logs, truncate > file, remove old archives. 6. Long-term: log rotation, monitoring, volume expansion or LVM resize.

### 78. A service is running but not responding to requests. How do you debug?

1. systemctl status service (any errors). 2. ss -tlnp (is port listening?). 3. curl localhost:port (local connectivity). 4. iptables -L (firewall blocking?). 5. ulimit -n (file descriptors exhausted?). 6. journalctl -u service (app errors). 7. strace -p PID (stuck syscall?). 8. Check upstream dependencies (DB, cache).

### 79. Server unreachable after a kernel/network config change. How do you recover?

If SSH: try console access (IPMI/KVM/cloud console). Check if networking service failed (ip addr show, networkctl). Revert last change (git or file backup). If iptables locked you out: at/batch job to flush rules was not set? Use emergency mode or recovery kernel. Document change management process to prevent recurrence.

### 80. How do you handle a zombie process accumulation?

Zombie = child exited, parent hasn't called wait(). Find with ps aux | grep Z. Kill parent process to have init(1) adopt and reap zombies. If parent is a critical service, fix the application to properly handle SIGCHLD. Mass zombies indicate a bug in the parent; don't just kill zombies.

### 81. Memory leak detected in a production Java application. Your approach?

1. Confirm: free -h, jstat -gc PID (heap usage trend). 2. Heap dump: jmap -dump:live,format=b,file=heap.hprof PID. 3. Analyze: MAT/VisualVM/Eclipse MAT for leak suspects. 4. GC logs enabled? Check for Full GC frequency. 5. Mitigation: restart with increased heap, rolling restart. 6. Root cause: analyze dump, fix code, redeploy.

### 82. How do you safely reduce I/O wait on a database server?

1. Confirm with iostat -x 1 (await, util). 2. Identify culprit: iotop -o. 3. Check if sequential vs random I/O (fio benchmark). 4. Tune: vm.dirty_ratio, I/O scheduler (mq-deadline for HDD, none for NVMe). 5. Move to faster storage (SSD/NVMe). 6. Add caching layer. 7. Optimize queries to reduce I/O. 8. Consider RAID striping.

### 83. A cron job is not running. How do you troubleshoot?

1. crontab -l (is it there?). 2. grep CRON /var/log/syslog (cron daemon running?). 3. Check PATH in crontab (different from user shell). 4. Script permissions and shebang correct? 5. Redirect output: * * * * * /script.sh >> /tmp/cron.log 2>&1. 6. Mailbox: does user have mail with error? 7. SELinux/AppArmor blocking the script?

### 84. How would you set up a high-availability NFS server?

Options: 1. Pacemaker/Corosync + NFS (DRBD for shared storage or SAN). 2. GlusterFS/CephFS as distributed filesystem. 3. Cloud: EFS (AWS), Filestore (GCP), Azure Files. For on-prem: two nodes with heartbeat, floating IP (VIP), shared LVM on SAN, export same path from active node. Monitor with drbd-overview, pcs status.

### 85. Explain how you would tune Linux for a high-traffic web server.

sysctl: somaxconn=65535, tcp_max_syn_backlog=65535, ip_local_port_range='1024 65535', tcp_tw_reuse=1, rmem_max/wmem_max=16MB. ulimits: nofile=1000000. Network: ethtool -G eth0 rx/tx 4096 (ring buffers), irqbalance, RSS/multi-queue NIC. Filesystem: noatime mount option. Disable THP if latency is critical. Enable BBR TCP congestion control.

# Bash Scripting

## BASIC — Syntax, Variables & Control Flow

### 1. What is a shebang line and why is it important?

#!/bin/bash (or #!/usr/bin/env bash) tells the kernel which interpreter to use. env version is more portable across systems where bash may not be in /bin. Without it, scripts may run with /bin/sh (dash) which lacks bash features. Make executable: chmod +x script.sh.

### 2. Explain variable types in Bash.

No type declarations by default — everything is a string. Declare types with: declare -i (integer), declare -a (array), declare -A (associative array), declare -r (readonly), declare -x (export to environment). Local variables in functions: local var=value.

### 3. What is the difference between $() and backticks for command substitution?

Both capture command output. $() is preferred: it's nestable $(cmd $(nested)), more readable, and handles quotes better. Backticks `cmd` are legacy POSIX syntax. Example: DATE=$(date +%Y%m%d) is preferred over DATE=`date +%Y%m%d`.

### 4. How does error handling work in Bash?

$? holds last exit code (0=success, non-zero=error). set -e exits on any error. set -u treats unset variables as errors. set -o pipefail makes pipelines fail if any component fails. set -x enables debug tracing. Combine: set -euo pipefail for robust scripts.

### 5. Explain the difference between [ ], [[ ]], and (( )).

[ ] is POSIX test command — use for portability. [[ ]] is bash built-in — supports regex (=~), no word splitting, safer with variables. (( )) is arithmetic evaluation — C-style expressions, 0=false, non-zero=true. Example: [[ $var =~ ^[0-9]+$ ]] for regex test.

### 6. What are positional parameters and special variables?

$0=script name, $1-$9 (args), ${10}+ for >9. $#=arg count, $@=all args (separate words), $*=all args (one word), $?=last exit code, $$=current PID, $!=last background PID, $-=shell options flags.

### 7. How do you handle arrays in Bash?

arr=(a b c); echo ${arr[0]}; echo ${arr[@]}; echo ${#arr[@]}; arr+=(d). Slicing: ${arr[@]:1:2}. Delete element: unset arr[1]. Associative: declare -A map; map[key]=val; echo ${map[key]}; echo ${!map[@]} (keys).

### 8. Explain string manipulation in Bash.

${#var} (length), ${var^^} (uppercase), ${var,,} (lowercase), ${var:pos:len} (substring), ${var/old/new} (replace first), ${var//old/new} (replace all), ${var#prefix} (strip prefix), ${var%suffix} (strip suffix), ${var##longest prefix}, ${var%%longest suffix}.

### 9. How does process substitution work?

<(cmd) creates a FIFO providing command output as a file. >(cmd) sends output to a command via FIFO. Useful for: diff <(sort file1) <(sort file2); while read line; do ...; done < <(command). Avoids subshell variable scope issues with pipes.

### 10. What is the difference between source (.) and executing a script?

source (.) runs the script in the current shell — environment changes (cd, exports, vars) persist. Executing (./script.sh or bash script.sh) spawns a subshell — changes don't affect parent. Use source for scripts that should modify current environment (.bashrc, virtualenv activate).

### 11. Explain here documents (heredoc) and here strings.

Heredoc: command << EOF ... EOF — multi-line input to command. <<- EOF strips leading tabs. <<'EOF' prevents variable expansion. Herestring: command <<< 'string' for single-line input. Example: mysql -u root << EOF\nSELECT 1;\nEOF

### 12. How do you trap signals in Bash?

trap 'cleanup_function' EXIT INT TERM. Common pattern: trap 'rm -f $tmpfile; exit' EXIT INT TERM. Cleanup runs on normal exit, Ctrl+C, or kill. trap '' SIGINT ignores signal. trap - SIGINT resets to default. Essential for scripts creating temp files or acquiring locks.

### 13. What is word splitting and globbing, and how to prevent them?

Bash splits unquoted variables on IFS (space/tab/newline) and expands * ? [] as globs. Prevent by quoting: "$var" prevents splitting/globbing. Use "$@" not $@. IFS=$'\n\t' changes splitting chars. set -f disables globbing. Unquoted variables in conditionals are a common bug source.

### 14. Explain conditional expressions and test operators.

Numeric: -eq -ne -lt -gt -le -ge. String: = (== in [[), != -z (empty) -n (non-empty). File: -e (exists) -f (file) -d (dir) -r -w -x (perms) -s (non-empty) -L (symlink). Logical: && || ! in [[]], -a -o in []. Use man test for full list.

### 15. How do you read files line by line in Bash?

while IFS= read -r line; do echo "$line"; done < file.txt. IFS= prevents stripping leading/trailing whitespace. -r prevents backslash interpretation. Avoid for line in $(cat file) — it word-splits. For CSV: IFS=',' read -ra fields <<< "$line" inside the loop.

### 16. What are Bash functions and how do local variables work?

function name() { local var=val; return 0; }. Local restricts variable to function scope. Functions inherit parent variables. Return only sets exit code (0-255); use echo/global var or declare -g for return values. Functions can be exported with declare -fx func.

### 17. How do you handle default values for variables?

${var:-default} — use default if var is unset or empty. ${var:=default} — assign default if unset/empty. ${var:?error msg} — error if unset/empty. ${var:+alternate} — use alternate if set. Pattern: : ${DEPLOY_ENV:=production} sets default without echo output.

### 18. Explain job control in Bash.

& — run in background. jobs — list background jobs. fg %1 — bring to foreground. bg %1 — continue in background. wait — wait for all background jobs. wait $pid — wait for specific PID. disown %1 — detach from shell. $! — PID of last background job. Ctrl+Z suspends current job.

### 19. What is IFS and how to use it?

Internal Field Separator determines word splitting (default: space, tab, newline). Temporarily change: IFS=':' read -ra parts <<< "$PATH". Reset: IFS=$' \t\n'. Use in loops to split CSV: while IFS=',' read -r a b c; do ... done < file.csv. Always restore IFS after changing.

### 20. How do you perform arithmetic in Bash?

$(( expr )) for arithmetic substitution, (( expr )) for arithmetic commands. let var=expr. declare -i forces integer type. bc -l for floating point: echo '3.14*2' | bc -l. awk for complex math: awk 'BEGIN{print 3.14*2}'. Bash integers only; use Python/awk for floats.

### 21. What is the difference between exit and return?

exit N terminates the script (or shell) with code N. return N exits from a function with code N. If used outside a function, return exits the sourced script. exit is for scripts/shells; return is for functions. Trap EXIT fires on exit but not return (unless sourced).

### 22. How do you check if a command exists in Bash?

command -v cmd >/dev/null 2>&1 (preferred, POSIX). type cmd &>/dev/null (bash built-in). which cmd (may not find functions/aliases). Pattern in scripts: if ! command -v docker &>/dev/null; then echo 'docker not found'; exit 1; fi.

### 23. Explain Bash regex with =~ operator.

[[ string =~ pattern ]] — ERE matching. No quotes around pattern (or use variable). BASH_REMATCH[0]=full match, [1]=first capture group. Example: if [[ $email =~ ^[^@]+@[^@]+\.[^@]+$ ]]; then echo valid; fi. Portability: use grep -E or sed -E for POSIX portability.

### 24. How do you handle multi-line strings in Bash?

Heredoc: var=$(cat << 'EOF'\nline1\nline2\nEOF). Printf: printf '%s\n' 'line1' 'line2'. $'...': var=$'line1\nline2'. Literal newline in quotes. Use readarray -t arr < <(command) to read into array. IFS=$'\n' mapfile -t arr < file for file to array.

### 25. What are Bash options (set -e, set -u, etc.) and their impacts?

set -e: exit on error (watch: compound commands, ||, &&). set -u: error on undefined variables. set -o pipefail: catch pipe failures. set -x: debug trace. set -v: verbose (prints input). set -n: parse without executing (syntax check). set -E: traps inherited by functions. Best practice: set -euo pipefail at top.

## INTERMEDIATE — Text Processing, Functions & Scripting Patterns

### 26. How do you use awk for data processing?

awk 'BEGIN{...} pattern{action} END{...}' file. Built-ins: NR (record num), NF (field count), $1-$NF (fields), FS (separator). Examples: awk '{print $1,$3}' (fields), awk -F: '{sum+=$3}END{print sum}' (sum), awk 'NR>1{print}' (skip header), awk '$3>100' (filter). Powerful for log parsing and report generation.

### 27. Explain sed usage for text transformation.

sed 's/old/new/g' (substitute), sed -n '5,10p' (print lines 5-10), sed '/pattern/d' (delete), sed -i 's/old/new/g' file (in-place). Address ranges: sed '2,5d', sed '/start/,/end/d'. Append: sed '/pattern/a\new line'. sed -E for extended regex. Always backup before -i: sed -i.bak.

### 28. How do you use find with complex expressions?

find /path -type f -name '*.log' -mtime +30 -delete. Combine: -and/-or/-not. Exec: -exec rm {} \; or -exec rm {} + (batch). Size: -size +100M. Permissions: -perm /u+s. User: -user root. -prune to skip dirs. Combine find with xargs for efficiency: find . -name '*.py' | xargs grep 'import os'.

### 29. Write a script to rotate log files keeping last 7 days.

find /var/log/app -name '*.log' -mtime +7 -delete; or use logrotate conf. Script: for f in /var/log/app/*.log.$(date -d '8 days ago' +%Y%m%d)*; do rm -f "$f"; done. Best practice: use logrotate with /etc/logrotate.d/app config specifying rotate 7, daily, compress, missingok, notifempty.

### 30. How do you process JSON in Bash?

Use jq tool: jq '.key' file.json, jq '.[] | select(.status=="active")' , jq -r '.name' (raw output). For simple cases: python3 -c "import json,sys; d=json.load(sys.stdin); print(d['key'])" < file.json. Avoid regex on JSON — always use proper parser. jq is the industry standard for shell JSON processing.

### 31. Explain xargs and its common use cases.

xargs reads stdin and builds command arguments. xargs -n1 (one arg per command), xargs -P4 (4 parallel), xargs -I{} (placeholder), -0 with find -print0 (handle spaces). Example: find . -name '*.tmp' -print0 | xargs -0 rm -f. Parallel: cat urls.txt | xargs -n1 -P10 curl -O. Safer than $(cmd) for large outputs.

### 32. How do you write a daemon-like script in Bash?

#!/bin/bash\nset -euo pipefail\n# Daemonize\nnohup bash -c 'while true; do do_work; sleep 60; done' > /var/log/daemon.log 2>&1 &\necho $! > /var/run/daemon.pid. Better approach: write a systemd unit file with Restart=on-failure and Type=simple. Pure Bash daemons are fragile; use systemd for production.

### 33. How do you implement retry logic in Bash?

retry() { local n=0 max=3 delay=5; until [ $n -ge $max ]; do "$@" && return; n=$((n+1)); echo "Attempt $n failed, retrying in ${delay}s"; sleep $delay; done; return 1; }. Usage: retry curl -f https://api/endpoint. Add exponential backoff: delay=$((delay*2)). Essential for network operations, API calls, flaky tests.

### 34. Explain how to use named pipes (FIFOs) in Bash.

mkfifo /tmp/mypipe. Producer: cmd1 > /tmp/mypipe &. Consumer: cmd2 < /tmp/mypipe. Unlike regular pipes, named pipes persist in filesystem, allow non-parent/child processes to communicate, and enable complex IPC patterns. Used in logging architectures and inter-process communication.

### 35. How do you lock a script to prevent concurrent execution?

Using flock: exec 9>/var/lock/myscript.lock; flock -n 9 || { echo 'Already running'; exit 1; }. Or: if [ -f /var/run/script.pid ] && kill -0 $(cat /var/run/script.pid) 2>/dev/null; then exit 1; fi; echo $$ > /var/run/script.pid. flock is more robust as it's automatically released on exit.

### 36. Write a Bash function to parse command-line arguments.

Use getopts for POSIX: while getopts 'f:v' opt; do case $opt in f) file=$OPTARG;; v) verbose=1;; esac; done. For long options: use getopt (GNU): opts=$(getopt -o f:v --long file:,verbose -- "$@"); eval set -- "$opts". For complex CLIs, consider using a dedicated argument parsing library or Python/Go.

### 37. How do you handle concurrent Bash scripts safely?

Use flock for mutual exclusion, mktemp for temp files (trap cleanup), atomic operations where possible (mv is atomic within same filesystem). For parallel processing: GNU parallel or xargs -P. Avoid race conditions by using $$ in temp filenames and always trapping EXIT.

### 38. How do you colorize output in Bash scripts?

ANSI codes: \033[0m (reset), \033[31m (red), \033[32m (green), \033[33m (yellow), \033[34m (blue), \033[1m (bold). Example: RED='\033[0;31m'; NC='\033[0m'; echo -e "${RED}Error${NC}". Check if terminal supports color: [ -t 1 ] && tput colors >/dev/null 2>&1. tput is more portable.

### 39. Explain Bash completion and how to write custom completions.

Bash completion scripts define how tab-completion works. In /etc/bash_completion.d/ or ~/.bash_completion. Use complete -F _function cmd. _function uses COMP_WORDS, COMP_CWORD, COMPREPLY. Example: complete -F _my_app myapp. compgen -W 'opt1 opt2' -- "${word}" generates matches. Major tools ship with completion scripts.

### 40. How do you perform network operations in Bash?

curl -sf URL (silent, fail on error), -o file (download), -H 'header: val' (headers), -d @data.json (POST), -L (follow redirects), --retry 3. nc (netcat) for raw connections. Check port: nc -zv host port. wget for downloads. Use curl -w '%{http_code}' for status codes. Avoid raw /dev/tcp except for simple checks.

### 41. Write a script to monitor a URL and alert if down.

while true; do code=$(curl -s -o /dev/null -w '%{http_code}' https://example.com); if [ "$code" != '200' ]; then echo "Alert: site down (HTTP $code)" | mail -s 'Site Down' ops@company.com; fi; sleep 60; done. Production: use Prometheus + Alertmanager, Nagios, or Datadog instead of custom scripts.

### 42. Explain Bash subshells and their implications.

( cmds ) creates a subshell — separate env, cd/variable changes don't affect parent. Pipes create subshells: while read line; done < <(cmd) avoids pipe subshell for variable scope. $() is also a subshell. Implications: heavy subshell use is slow (fork overhead), variable changes are lost. Use functions instead where possible.

### 43. How do you securely handle sensitive data in Bash scripts?

Never hardcode passwords. Use: environment variables (set by calling process), secret managers (Vault, AWS SSM), prompt with read -rs password (silent), /dev/stdin. Clean up: unset password after use. Avoid: echo password | cmd (visible in /proc), passing as args (visible in ps). Use install -m 600 for sensitive files.

### 44. What is the difference between printf and echo?

printf is more portable and predictable: printf '%s\n' "$var" won't interpret escape sequences in $var. echo -e interprets escapes but behavior varies across shells. printf supports format strings like C printf. Use printf for scripts; echo for interactive use. printf '%s\0' for null-separated output.

### 45. How do you profile Bash script performance?

bash -x script.sh 2> trace.log (traces all commands). time script.sh (elapsed time). PS4='$(date +%s%3N) ' bash -x script.sh (timestamps per command). For hotspots: time each function. Avoid unnecessary subshells, external commands in loops (read from files directly), and repeated command substitutions. Use [[ not [ for speed.

### 46. How do you manage script dependencies and portability?

Check required tools at start: for cmd in jq curl aws; do command -v $cmd >/dev/null || { echo "$cmd required"; exit 1; }; done. Use #!/usr/bin/env bash. Avoid bashisms if targeting /bin/sh. Use shebang with proper version if needed. Document OS/version requirements. Container packaging ensures dependencies.

### 47. Explain mapfile/readarray and when to use them.

mapfile -t arr < file reads file into array (one element per line). readarray is synonym. mapfile -t arr < <(cmd) reads command output. Much faster than while read loops for large files. -n limit: mapfile -n 100 (first 100 lines). -O offset: start filling at index. Use when you need random access to all lines.

### 48. How do you write idempotent Bash scripts?

Check before creating: [ -f /etc/app.conf ] || create_config. Verify before deleting: [ -d /tmp/work ] && rm -rf /tmp/work. Use --create-dirs with curl. Idempotent installs: apt-get install -y pkg (safe to re-run). Create lock/sentinel files to track completed steps. Test scripts can run multiple times without side effects.

### 49. What are the best practices for script logging?

Use structured logging: log() { echo "$(date -u +%Y-%m-%dT%H:%M:%SZ) [$1] $2"; }. Log levels: DEBUG/INFO/WARN/ERROR. Redirect stderr: exec 2> /var/log/script.err. Tee for simultaneous stdout+log: exec > >(tee -a /var/log/script.log). Include PID, timestamp, level. Rotate logs. Send critical alerts to monitoring.

### 50. How do you unit test Bash scripts?

Use bats-core (Bash Automated Testing System): @test 'description' { run my_function arg; assert_success; assert_output 'expected'; }. Or shunit2. shellcheck for static analysis (run in CI). Test functions in isolation by sourcing script and calling functions. Mock external commands by temporarily overriding PATH with a test dir containing mock scripts.

## ADVANCED & TROUBLESHOOTING

### 51. A Bash script is consuming too much memory. How do you debug?

Use /usr/bin/time -v ./script.sh for peak memory. Monitor with watch -n1 'ps -o pid,vsz,rss -p PID'. Common causes: accumulating large arrays (unset after use), command substitution capturing huge output, while loops building strings. Fix: stream processing instead of buffering, process files in chunks, use awk/sed instead of bash string ops for large data.

### 52. How do you handle signals properly in a script that spawns children?

trap 'kill -- -$$; exit' INT TERM. kill -- -$$ sends signal to entire process group. Or: trap 'kill $child_pid; exit' INT TERM; cmd &; child_pid=$!; wait. With multiple children: pids=(); for ...; do cmd & pids+=($!); done; trap 'kill ${pids[@]}' EXIT; wait ${pids[@]}. Always wait for cleanup to complete.

### 53. A cron script fails silently. How do you diagnose?

1. Run script manually as same user (sudo -u cronuser). 2. Check PATH: cron has minimal env (echo $PATH differs). 3. Redirect all output: script.sh > /tmp/out.log 2>&1. 4. grep CRON /var/log/syslog for

cron daemon errors. 5. MAILTO=youremail@domain in crontab header. 6. Add env command at start of cron script to log environment. Most cron issues are PATH or env problems.

### 54. Explain how to implement a pipeline in Bash with error propagation.

set -o pipefail alone is insufficient for detecting which command failed. Use: cmd1 | cmd2; echo ${PIPESTATUS[@]}. PIPESTATUS is array of each pipeline exit code. Or with named pipes and explicit error checking. For complex pipelines, consider using explicit temp files or process substitution with error checking at each stage.

### 55. How do you write a script that safely deploys code and rolls back on failure?

deploy() { backup_dir=/deploy/backup/$(date +%s); cp -r $APP_DIR $backup_dir; trap 'rollback $backup_dir' ERR; git pull && systemctl restart app || return 1; trap - ERR; }; rollback() { cp -r $1/. $APP_DIR; systemctl restart app; }. Use atomic symlink swap: ln -snf new_version current for zero-downtime. Always test rollback procedure.

### 56. A Bash script is taking 10x longer than expected. How do you find the bottleneck?

Enable timestamps in PS4: PS4='+$(date +%s%3N) ' bash -x script.sh 2>trace.log. Then: awk '{print $1}' trace.log | uniq -d to find slow lines. Common culprits: external commands in loops (replace with awk/built-ins), repeated subshells, cat | grep (use grep directly), sleep in polling loops, excessive file reads. Profile each section with time { ... }.

### 57. How do you implement distributed locking across multiple servers?

Redis-based: use redis-cli SET lockkey $hostname NX EX 300 (atomic, expires). Check: if [ $(redis-cli SET lock $host NX EX 300) = 'OK' ]; then do_work; redis-cli DEL lock; fi. Alternative: shared NFS file with flock, or etcd. For cron jobs: use a distributed cron system (Kubernetes CronJob, Rundeck) that handles locking natively.

### 58. How do you handle encoding issues (UTF-8 etc.) in Bash scripts?

export LANG=en_US.UTF-8 LC_ALL=en_US.UTF-8 at script start. Use -r with read to prevent mangling. iconv for conversion: iconv -f latin1 -t utf-8 file. file command detects encoding. od -c to inspect raw bytes. grep -P for Perl regex with Unicode. bash handles UTF-8 in [[ ]] comparisons. ${#var} counts chars, not bytes in UTF-8.

### 59. Explain advanced heredoc patterns for generating config files.

cat > /etc/app.conf << EOF\nhost=${DB_HOST}\nEOF — variables expand. cat > /etc/app.conf << 'EOF' — no expansion (for scripts with special chars). Indent: cat > file <<- EOF (tabs stripped, not spaces). Use envsubst for template substitution: envsubst < template.conf > /etc/app.conf. For complex configs, use Ansible templates or sed.

### 60. How do you write a Bash script that's both a library and executable?

Pattern: if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then main "$@"; fi. This runs main() when executed directly but not when sourced. Functions are available when sourced. Similar to Python's if __name__ == '__main__'. Useful for scripts that other scripts source for shared functions.

## □ REAL-WORLD SCENARIOS

### 61. Write a deployment script for a blue-green deploy.

Script: set -euo pipefail. Determine inactive env (if active=blue, use green). Deploy to inactive: update code, restart service on inactive. Health check: retry_curl http://inactive-env/health. Switch load balancer: update nginx upstream or DNS. Keep old env running for 10 min then stop. Rollback: switch LB back instantly. Log all steps with timestamps.

### 62. How would you script automated SSL certificate renewal?

Use certbot renew --pre-hook/--post-hook or deploy-hook. Custom script: check expiry with: openssl x509 -in cert.pem -noout -checkend 2592000 (30 days). If expiring: call ACME API or certbot renew. Post-renewal hook: systemctl reload nginx. Run daily via cron or systemd timer. Alert if renewal fails. Test with --dry-run.

### 63. Script to parse Nginx access logs and report top 10 IPs by request count.

awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -10. Extend: awk '$9~/^[45]/' (filter errors), awk '{sum[$1]+=$10}END{for(i in sum)print sum[i],i}' | sort -rn | head (bytes per IP). Schedule with cron, output to report. Add rate-limiting for top IPs: awk+ipset or fail2ban.

### 64. How would you automate database backups with rotation?

#!/bin/bash\nset -euo pipefail\nBAK_DIR=/backups\nDATE=$(date +%Y%m%d_%H%M)\nmysqldump -u$USER -p$PASS mydb | gzip > $BAK_DIR/db_$DATE.sql.gz\nfind $BAK_DIR -name 'db_*.sql.gz' -mtime +7 -delete\n# Upload to S3\naws s3 cp $BAK_DIR/db_$DATE.sql.gz s3://mybucket/db-backups/\n# Alert if backup size suspiciously small\nsize=$(stat -c%s $BAK_DIR/db_$DATE.sql.gz); [ $size -lt 1000 ] && alert 'Backup too small'

### 65. Script to monitor disk usage and alert when threshold exceeded.

#!/bin/bash\nTHRESHOLD=85\nwhile IFS= read -r line; do\n  usage=$(echo $line | awk '{print $5}' | tr -d '%')\n  mount=$(echo $line | awk '{print $6}')\n  if [ $usage -ge $THRESHOLD ]; then\n msg="ALERT: $mount at ${usage}% on $(hostname)"\n    echo $msg | mail -s 'Disk Alert' ops@company.com\n    # or: curl -X POST slack_webhook -d "{\"text\":\"$msg\"}"\n  fi\ndone < <(df -h | tail -n +2)

### 66. How would you write a script to manage application config across environments?

Use environment-specific config files: config/base.conf, config/prod.conf, config/staging.conf. Script merges: merge_config() { cat config/base.conf config/${ENV}.conf > /etc/app.conf; }. Or use envsubst with templates. Better: use Ansible vars, Helm values, or Consul/Vault for production. Script validates config schema before applying: python3 -c 'import configparser; configparser.ConfigParser().read("file.conf")'.

### 67. Write a health check script for a microservice.

#!/bin/bash\ncheck_http() { curl -sf --max-time 5 $1 >/dev/null; }\ncheck_port() { nc -zw3 $1 $2; }\ncheck_disk() { df / | awk 'NR==2{print $5}' | tr -d '%' | xargs -I{} test {} -lt 90; }\ncheck_mem() { free | awk '/Mem/{print int($3/$2*100)}' | xargs -I{} test {} -lt 90; }\n\nfail=0\ncheck_http http://localhost:8080/health || { echo 'HTTP FAIL'; fail=1; }\ncheck_port localhost 5432 || { echo 'DB FAIL'; fail=1; }\ncheck_disk || { echo 'DISK FAIL'; fail=1; }\nexit $fail

### 68. Script to perform rolling restart of application instances.

instances=(app01 app02 app03)\nfor host in "${instances[@]}"; do\n  echo "Restarting $host"\n  # Remove from LB\n  lb_remove $host\n  # Wait for connections to drain\n  sleep 10\n  # Restart service\n  ssh $host 'systemctl restart myapp'\n  # Health check\n  retry_curl http://$host:8080/health || { echo "$host unhealthy!"; exit 1; }\n  # Return to LB\n  lb_add $host\n  echo "$host back in rotation"\ndone

### 69. How would you script Kubernetes namespace cleanup for stale preview environments?

#!/bin/bash\nfor ns in $(kubectl get ns -o name | grep 'preview-' | cut -d/ -f2); do\n  created=$(kubectl get ns $ns -o jsonpath='{.metadata.creationTimestamp}')\n  age=$(( ($(date +%s) - $(date -d $created +%s)) / 86400 ))\n  if [ $age -gt 7 ]; then\n    echo "Deleting stale namespace: $ns (${age}d old)"\n kubectl delete ns $ns --wait=false\n  fi\ndone

### 70. Write a Bash script for automated application on-boarding.

Script: create_app() { APP=$1; GROUP=$2; mkdir -p /app/$APP/{config,logs,data}; useradd -r -s /sbin/nologin $APP; chown -R $APP:$GROUP /app/$APP; cat > /etc/systemd/system/$APP.service << EOF [Unit]... [Service] User=$APP ExecStart=/app/$APP/bin/start... EOF; systemctl daemon-reload; systemctl enable $APP; echo "App $APP on-boarded"; }. Include: firewall rules, monitoring registration, log rotation setup.

# Docker

## BASIC — Images, Containers & Dockerfile

### 1. What is Docker and how does it differ from a VM?

Docker uses OS-level virtualization (namespaces + cgroups) sharing the host kernel — lightweight, fast startup (~ms). VMs virtualize hardware with a full OS per VM — stronger isolation, heavier (~GBs, slow boot). Docker containers are portable and image-based. VMs better for different OS kernels or hard isolation requirements.

### 2. Explain the Docker architecture.

Docker daemon (dockerd) — manages containers, images, volumes. Docker CLI — sends commands to daemon via REST API. containerd — container runtime managing lifecycle. runc — OCI runtime creating containers. Registry — stores images. Docker Desktop adds a hypervisor VM on Mac/Windows to run the Linux daemon.

### 3. What is a Docker image and how is it structured?

An image is a read-only layered filesystem. Each Dockerfile instruction creates a layer. Layers are shared across images (content-addressable via SHA256). Union filesystem (overlay2 on Linux) merges layers. A container adds a writable layer on top. Images are identified by repository:tag and digest.

### 4. Explain Dockerfile best practices.

Use official minimal base images (alpine, distroless). Order instructions by change frequency (deps before code). Combine RUN commands to minimize layers. Use .dockerignore. Multi-stage builds. Copy only needed files. Don't run as root (USER instruction). Use COPY not ADD unless extracting archives. Pin versions. Use BuildKit.

### 5. What is the difference between CMD and ENTRYPOINT?

ENTRYPOINT defines the executable; CMD provides default args. ENTRYPOINT ['nginx'] + CMD ['-g', 'daemon off;'] = nginx -g 'daemon off;'. Override CMD with docker run args. Override ENTRYPOINT with --entrypoint. Shell form (CMD nginx) runs via /bin/sh -c — doesn't receive signals properly. Exec form (CMD ['nginx']) is preferred.

### 6. How do Docker volumes work and when to use bind mounts vs volumes?

Volumes are managed by Docker (/var/lib/docker/volumes/) — portable, backed up easily, work with volume drivers. Bind mounts map a host path into container — for development (live code) or accessing host files. tmpfs mounts are in-memory only. Use volumes for data persistence in production, bind mounts for dev workflows.

### 7. What is Docker networking and explain bridge, host, none modes?

bridge (default): containers on virtual network, isolated from host, NAT for external. host: container shares host network stack — no isolation, better performance. none: no networking. overlay: multi-host (Swarm/Compose). macvlan: container gets its own MAC/IP. Network: docker network create, docker network connect.

### 8. Explain port publishing in Docker.

docker run -p hostport:containerport (docker manages iptables rules). -P publishes all EXPOSE'd ports to random host ports. EXPOSE in Dockerfile is documentation only — doesn't actually publish. --network=host uses host ports directly. docker port container to inspect mappings. Security: don't expose ports unnecessarily.

### 9. What is Docker Compose and when would you use it?

Compose defines and runs multi-container applications via docker-compose.yml. v1 (docker-compose CLI) vs v2 (docker compose plugin). Defines services, networks, volumes declaratively. Use for: local development, testing, simple deployments. For production at scale: use Kubernetes or Docker Swarm.

### 10. How do you reduce Docker image size?

Multi-stage builds (copy only artifacts). Use alpine/distroless base. Combine RUN layers. .dockerignore. Remove build deps in same RUN layer: apt-get install build-essential && build && apt-get remove build-essential. Use --no-install-recommends. Check layers: docker history image. Dive tool for layer inspection. Strip debug symbols.

### 11. What is a multi-stage Docker build?

Multiple FROM statements in one Dockerfile. Build stage compiles code (full SDK), final stage copies only artifacts (minimal runtime). Example: FROM golang:1.21 AS builder; RUN go build; FROM gcr.io/distroless/base; COPY --from=builder /app/binary /app. Result: final image has only binary, no Go toolchain.

### 12. Explain Docker's overlay2 storage driver.

overlay2 uses Linux OverlayFS merging multiple directories into one view. Lower dirs = image layers (read-only), upper dir = container layer (read-write), merged view = container sees. Efficient — unchanged files are not copied (copy-on-write). Default on modern kernels. Check: docker info | grep Storage.

### 13. How does Docker handle container logging?

Default json-file driver stores logs as JSON in /var/lib/docker/containers/ID/ID-json.log. docker logs container (tail, follow). Other drivers: syslog, journald, fluentd, awslogs, gelf, splunk. Set via --log-driver and --log-opt. Configure max size to prevent disk fill: --log-opt max-size=10m --log-opt max-file=3.

### 14. What is Docker Content Trust (DCT)?

DCT uses Notary to sign and verify images. DOCKER_CONTENT_TRUST=1 enforces signature verification. Images are signed with private key, verified with public key stored in Docker Hub. Ensures you only run trusted images. For production: use DCT or digest-based image references (image@sha256:...) to prevent tag mutability issues.

### 15. Explain Docker resource limits.

--memory 512m (RAM limit), --memory-reservation 256m (soft limit), --cpus 1.5 (CPU limit, fractional), --cpu-shares 512 (relative weight), --pids-limit 100 (PID limit), --blkio-weight 500 (I/O weight), --device-read-bps /dev/sda:10mb (I/O rate). In Compose: deploy.resources.limits. Kubernetes uses requests/limits.

### 16. How do you pass environment variables to containers?

docker run -e VAR=val, -e VAR (from host env), --env-file .env. In Compose: environment: or env_file:. Best practices: don't hardcode secrets in images, use -e at runtime, integrate with secret managers (Vault, AWS SSM, Kubernetes secrets). Inspect: docker inspect container | grep -A Env.

### 17. What is the difference between docker stop and docker kill?

docker stop sends SIGTERM, waits --time (default 10s) for graceful shutdown, then SIGKILL. docker kill sends SIGKILL (or specified signal) immediately. Always use docker stop for production to allow graceful shutdown. Configure timeout: docker stop -t 30. Application should handle SIGTERM to cleanup connections, flush buffers.

### 18. How do you update a container in production without downtime?

Use orchestration: Kubernetes rolling update, Docker Swarm update (--update-parallelism, --update-delay). Manually: bring up new container, wait for health check, switch traffic (update nginx upstream or load balancer), stop old container. Or blue-green: full replacement. Never: docker stop old; docker start new (causes downtime).

### 19. What is BuildKit and its advantages?

BuildKit (DOCKER_BUILDKIT=1, now default) offers: parallel builds, improved caching, new Dockerfile syntax (RUN --mount), build secrets (--secret), SSH forwarding (--ssh), inline cache, better performance. RUN --mount=type=cache,target=/root/.npm speeds builds by caching npm/pip dirs between builds.

### 20. Explain Docker security best practices.

Non-root user (USER 1001), read-only filesystem (--read-only), no privileged mode, drop capabilities (--cap-drop ALL --cap-add NET_BIND_SERVICE), use seccomp profiles, don't mount Docker socket, use distroless/minimal images, scan with Trivy/Snyk, sign images, network segmentation, no secrets in images/env (use secrets).

### 21. How do Docker health checks work?

HEALTHCHECK --interval=30s --timeout=10s --retries=3 CMD curl -f http://localhost/health || exit 1. States: starting, healthy, unhealthy. Docker Swarm and K8s use health status to route traffic and

restart containers. docker inspect shows health status. If unhealthy, Swarm automatically replaces the container.

### 22. Explain Docker's garbage collection and cleanup.

docker system prune (remove all unused: stopped containers, dangling images, unused networks, build cache). -a (all unused images). --volumes (include volumes). docker image prune, docker container prune, docker volume prune, docker builder prune. Set up cron job for cleanup on build servers. docker system df for space usage overview.

### 23. What is Docker Swarm and how does it compare to Kubernetes?

Swarm is Docker's native clustering/orchestration — simpler to set up, uses Docker Compose syntax, good for smaller deployments. K8s is more powerful, extensible, larger ecosystem, better for complex apps. Swarm: docker swarm init, docker service create. For new projects, Kubernetes is the industry standard. Swarm is being used less.

### 24. How does container networking with overlay networks work in Swarm?

Overlay networks span multiple Docker hosts using VXLAN encapsulation. Services communicate by name (DNS-based service discovery). Encrypted with --opt encrypted. Ingress network handles routing mesh (any node can receive traffic for any service). docker network create --driver overlay --attachable mynet.

### 25. Explain Docker secrets management.

Docker Swarm secrets: docker secret create mysecret file.txt; reference in service as /run/secrets/mysecret (in-memory tmpfs, not exposed in inspect or env). Kubernetes uses similar Secrets (base64 encoded, can integrate with external secret stores via CSI drivers). Vault agent injection for both platforms. Never use ENV for sensitive data in production.

## INTERMEDIATE — Compose, Registry & Production

### 26. How do you structure a production docker-compose.yml?

Separate concerns: docker-compose.yml (base), docker-compose.override.yml (dev), docker-compose.prod.yml (prod). Merge: docker-compose -f base.yml -f prod.yml up. Production: specify image tags (not build), resource limits (deploy.resources), restart policies, health checks, named volumes, external networks. Don't use 'latest' tag.

### 27. Explain Docker registry options and when to use each.

Docker Hub: public default, free public, paid private. ECR (AWS): integrated with IAM, lifecycle policies, immutable tags. GCR/AR (GCP): integrated with GKE. Harbor: self-hosted, RBAC, vulnerability scanning, image replication, proxying. Nexus/Artifactory: enterprise artifact management including Docker. Choose based on cloud provider and compliance requirements.

### 28. How do you implement image scanning in CI/CD?

Trivy (most popular, fast): trivy image myimage:tag in CI. Snyk: snyk container test. Docker Scout (integrated in Docker). Clair, Anchore. Integrate in Dockerfile build stage or as CI step. Fail pipeline on CRITICAL vulnerabilities. Set up periodic scanning of existing images. ECR, GCR have built-in scanning.

### 29. What is a Docker image tag strategy for production?

Never use latest in production. Use: semantic versioning (v1.2.3), git commit SHA (image:abc1234), build number (image:build-42). Immutable tags: once pushed, same tag always same image. Mutable tags: rolling. Best: image:v1.2.3-sha256shortid. Store full digest in deployment manifests. Policy: latest only for dev/testing.

### 30. How do you optimize Docker build caching?

Order Dockerfile instructions: rarely changed (OS, deps) → frequently changed (code). Separate dependency install from code copy: COPY package.json .; RUN npm install; COPY . . Only code changes invalidate npm layer. Use BuildKit cache mounts: RUN --mount=type=cache,target=/root/.npm npm install. Layer invalidation is all-or-nothing below changed line.

### 31. Explain Docker networking for microservices.

Services on same Compose network communicate by service name (DNS). Create separate networks for isolation: frontend-net (web+app), backend-net (app+db). App connects to both; web only to frontend-net. Use aliases for service discovery. Port 80 in container, expose only needed ports to host. Service mesh (Envoy/Istio) for advanced mTLS/observability.

### 32. How do you handle database migrations with Docker?

Options: 1. Init container runs migration before app (Kubernetes initContainers). 2. Entrypoint script: check migration status, run if needed, then start app. 3. Separate migration job (Kubernetes Job). 4. CI/CD pipeline step before deployment. Use migration tools (Flyway, Liquibase, Alembic) with version tracking. Always backup before migration. Test rollback procedure.

### 33. What is Docker's --init flag and why use it?

Adds tini as PID 1 in the container. Tini properly handles: zombie reaping (waitpid), signal forwarding to child processes. Without it, if your app is PID 1: it must handle zombie reaping (most apps don't), signals may not propagate correctly to child processes. Use --init or install tini in Dockerfile. Kubernetes has equivalent settings.

### 34. How do you set up a private Docker registry with Harbor?

Deploy Harbor (Helm chart for K8s or docker-compose). Configure: external URL, database, Redis, object storage. Set up HTTPS (cert-manager or custom cert). Configure auth (local, LDAP, OIDC). Create projects with pull/push policies. Set up image replication to backup registry. Configure vulnerability scanning (Trivy integration). Push: docker login harbor.company.com; docker tag; docker push.

### 35. Explain Docker's networking and how to debug connectivity issues.

docker network inspect <net> (subnets, connected containers). docker exec container ping other-container (name resolution). nslookup service-name (DNS in container). docker exec container curl http://service:port (HTTP). Check iptables: iptables -t nat -L (Docker adds DNAT rules). For overlay: docker run --rm --network=overlay-net nicolaka/netshoot for debug tools.

### 36. How do you implement Docker logging to a central system?

Use logging driver: --log-driver=fluentd --log-opt fluentd-address=localhost:24224, or awslogs, splunk, gelf. Or use log shipper sidecar (Fluentbit DaemonSet in K8s). Compose: services.app.logging.driver: fluentd. Structure logs as JSON for easy parsing. Add container metadata: service name, version, environment as structured fields.

### 37. How do you implement zero-downtime deployments with Docker?

With Compose: docker-compose up -d --scale app=2 (scale up), wait for health, docker-compose up -d --scale app=1 (scale down old). With Nginx: upstream round-robin, rolling update. With Swarm: docker service update --update-parallelism 1 --update-delay 10s. With K8s: rolling update strategy. Key: health checks and graceful shutdown handlers are essential.

### 38. Explain Docker content addressability and digests.

Every layer and image has SHA256 digest of its content. Image digest: docker inspect --format='{{index .RepoDigests 0}}' image. Pull by digest: docker pull image@sha256:abc123 — immutable, cannot be altered. Tag can point to different digest over time (mutable). Digests ensure you run exactly what you expect. Essential for reproducibility in production.

### 39. What are distroless images and when to use them?

Distroless images (Google's gcr.io/distroless) contain only the app runtime — no shell, package manager, or OS utilities. Benefits: smaller attack surface, smaller size, forces good practices. Drawback: harder to debug (no shell for exec). Use in production for security. Debug variant available: :debug tag includes busybox. Alternative: scratch (completely empty, for static binaries).

### 40. How do you handle Docker in a CI/CD pipeline?

Build: docker build with BuildKit, --cache-from previous image for speed. Test: docker run --rm test-image pytest. Scan: trivy image. Push to registry with commit SHA tag. Deploy: update image reference in K8s manifests (kustomize set image or Helm upgrade). Use DOCKER_BUILDKIT=1, layer caching, parallel builds. Consider Kaniko for in-cluster builds.

## ADVANCED & TROUBLESHOOTING

### 41. A Docker container keeps crashing. How do you debug?

docker logs container --tail 100; docker inspect container (exit code, OOMKilled, RestartCount); docker events --filter container=name; dmesg | grep docker (kernel OOM); docker stats (resource

limits); Run interactively: docker run -it --entrypoint /bin/sh image; docker exec into running container. Check health check. Review CMD vs ENTRYPOINT.

### 42. How do you debug network connectivity between Docker containers?

docker network inspect network-name. docker exec container ping service-name. docker run --rm --network=container:myapp nicolaka/netshoot (rich debug tools). Check DNS: docker exec container cat /etc/resolv.conf; docker exec container nslookup service. tcpdump on veth: find veth pair with ip link show | grep veth, then tcpdump -i veth0 on host.

### 43. Container is slow to stop (SIGTERM not working). How to fix?

Signal PID 1 issue. Check: is app PID 1 (docker top container)? Fix: use exec form ENTRYPOINT ['/app'] not shell form. Or add tini (--init flag or install tini). Application code must handle SIGTERM. Increase timeout: docker stop -t 30. Check for signal forwarding: strace -e trace=signal -p 1 (inside container). Compile signal handler in app.

### 44. How do you resolve Docker disk space issues on a build server?

docker system df (usage breakdown). docker system prune -af --volumes (cleanup all unused). Set daemon config: /etc/docker/daemon.json: 'log-driver': 'json-file', 'log-opts': {'max-size':'10m'}. Set up cron: 0 2 * * * docker system prune -f. Configure BuildKit cache limits. Move /var/lib/docker to larger volume. Analyze with dive tool.

### 45. Explain and fix 'no space left on device' in Docker builds.

Could be: overlay2 storage full (df -h /var/lib/docker), inode exhaustion (df -i), too many layers. Fix: docker system prune, increase disk. Inode fix: clean up many small files. Layer limit (127 for overlay2): combine RUN statements. BuildKit generally doesn't hit layer limits. Check /var/lib/docker/tmp for orphaned build files.

### 46. How would you troubleshoot a memory leak in a Docker container?

docker stats container (monitor MEM USAGE trend). docker exec container cat /proc/meminfo. If JVM: docker exec container jmap -histo PID > histo.txt. Enable jvm flags: -XX:+HeapDumpOnOutOfMemoryError. For native leaks: valgrind or heaptrack (needs debug image). Set memory limits to trigger OOM: --memory 512m. Check /sys/fs/cgroup/memory/memory.oom_control. Analyze OOM in dmesg.

### 47. How do you secure the Docker daemon?

Use TLS for daemon socket (dockerd --tls flags or in /etc/docker/daemon.json). Restrict socket access (docker group — anyone in docker group is effectively root). Use rootless Docker (dockerd-rootless). Enable user namespaces (userns-remap). Disable legacy APIs. Audit: docker bench for security (CIS Docker Benchmark tool). Don't expose daemon on network without TLS.

### 48. Explain Docker's user namespace remapping.

userns-remap in /etc/docker/daemon.json maps container root (UID 0) to an unprivileged host user. Container UID 0 = host UID 100000 (for example). Mitigates container escape risk — even if attacker

escapes container, they're an unprivileged host user. Trade-off: some image permission issues, no shared UID with host. Enable: 'userns-remap': 'default' in daemon.json.

### 49. How do you implement image promotion across environments?

Pattern: build once, promote through registries/tags. Build: image:sha-abc123. Test in dev, tag as :staging, :prod when approved. docker tag image:sha registry/app:prod; docker push. Never rebuild image for different env — pass env-specific config at runtime via env vars or mounted configs. Use registry namespaces: registry/dev/app:sha → registry/prod/app:sha.

### 50. How do you handle Docker container configuration management at scale?

For large-scale: use Kubernetes ConfigMaps/Secrets, Helm values, or Kustomize overlays per environment. For simpler setups: docker-compose with .env files per environment. Use Consul/Vault for dynamic config. Pattern: runtime config via env vars, secrets via mounted files from secret store. Templating: envsubst, gomplate, consul-template. IaC: Terraform for infrastructure, Helm for K8s app config.

## □ REAL-WORLD SCENARIOS

### 51. Design a Docker-based CI/CD pipeline for a Node.js app.

Pipeline: 1. Checkout code. 2. Build: DOCKER_BUILDKIT=1 docker build --cache-from registry/app:cache -t app:$CI_COMMIT_SHA. 3. Test: docker run --rm app:$CI_COMMIT_SHA npm test. 4. Security scan: trivy image app:$CI_COMMIT_SHA --exit-code 1 --severity CRITICAL. 5. Push: docker push registry/app:$CI_COMMIT_SHA + :latest. 6. Deploy: update K8s manifest image tag. 7. Notify Slack.

### 52. How would you migrate a monolith to Docker containers?

1. Inventory: dependencies, ports, file paths, env vars. 2. Dockerize: create Dockerfile, docker-compose for local dev. 3. Externalize state: move DB, sessions, files to external services. 4. Config via env vars (12-factor). 5. Add health endpoints. 6. Build pipeline. 7. Deploy alongside VM (traffic gradual shift). 8. Monitor, optimize. Break into microservices over time if needed. Don't do everything at once.

### 53. How do you set up Docker for a multi-region deployment?

Use registry replication (ECR cross-region, Harbor replication, or push to multiple regions). Deploy identical images per region. Use CDN for registry pulls. Infrastructure: same image, region-specific env vars (DB endpoints). Orchestration: EKS/GKE per region, Route53/Traffic Manager for DNS routing. Data: replicated databases (RDS Multi-Region, CockroachDB). Test each region independently.

### 54. A Docker container is getting OOMKilled repeatedly. What do you do?

1. Confirm: docker inspect container | grep OOMKilled; dmesg | grep -i oom. 2. Profile: docker stats (what's the actual memory usage?). 3. Fix options: a) Increase --memory limit (quick fix). b) Fix memory leak in app (root cause). c) Add JVM -Xmx flag if Java. d) Add Node.js --max-old-space-size. 4. Monitor with Prometheus container_memory_working_set_bytes. 5. Set alert before OOM occurs.

### 55. How would you containerize a legacy application with hardcoded paths?

Use bind mounts to map container paths to expected locations: -v /data/config:/etc/legacy-app. Or create wrapper scripts: mkdir -p /expected/path && ln -s /actual /expected/path. Multi-stage build: first stage sets up path structure. Use ENV variables where possible. If truly inflexible: use a custom base image that sets up the expected filesystem structure. Document all path assumptions.

# GitHub Actions

## BASIC — Workflows, Jobs & Steps

### 1. What is GitHub Actions and its core concepts?

GitHub Actions is GitHub's CI/CD platform. Core concepts: Workflow (YAML file in .github/workflows/), Event (triggers: push, PR, schedule, manual), Job (set of steps on same runner), Step (individual task: action or command), Runner (VM/container executing jobs), Action (reusable unit, from marketplace or custom), Artifact (files passed between jobs).

### 2. Explain the structure of a GitHub Actions workflow file.

name: (optional display name), on: (triggers), env: (global env vars), jobs: (job definitions). Each job: runs-on, needs (dependencies), env, steps: (array of: uses/run, name, with, env, if). Indentation is critical (YAML). Files in .github/workflows/*.yml. Multiple workflow files are independent.

### 3. What events can trigger a GitHub Actions workflow?

push, pull_request (types: opened/synchronize/closed), schedule (cron), workflow_dispatch (manual), repository_dispatch (API), workflow_call (reusable), release, issues, issue_comment, create, delete, deployment, pull_request_review, check_run, status, registry_package. Filter with branches, paths, tags filters.

### 4. What is the difference between jobs.needs and job dependencies?

needs: [job1, job2] — current job waits for listed jobs to succeed. Creates a DAG. jobs without needs run in parallel. needs with if: always() runs even if dependency failed. Access outputs from dependency: ${{ needs.job1.outputs.myvalue }}. Circular dependencies are not allowed.

### 5. How do you use GitHub Actions expressions?

${{ expression }} in workflow YAML. Context objects: github (event data), env (env vars), job, steps, runner, secrets, needs, inputs. Operators: ==, !=, &&, ||, !, >, <. Functions: contains(), startsWith(), endsWith(), format(), join(), toJSON(), fromJSON(). Conditionals: if: ${{ github.ref == 'refs/heads/main' }}.

### 6. What are GitHub Actions contexts and how do you use them?

github: event, ref, sha, actor, repository, workflow. env: environment variables. job: status, services. steps: outputs and outcome of previous steps. runner: OS, temp, tool_cache. secrets: encrypted secrets. needs: outputs from dependency jobs. inputs: workflow_dispatch/workflow_call inputs. Access: ${{ github.actor }}, ${{ secrets.MY_SECRET }}.

### 7. How do you set and use environment variables in GitHub Actions?

Workflow-level: env: VAR: value. Job-level env: under jobs.job_name.env. Step-level: under steps.step_name.env. Dynamic: echo 'VAR=value' >> $GITHUB_ENV (available in subsequent steps). Job/step outputs: echo 'result=value' >> $GITHUB_OUTPUT. Secrets: ${{ secrets.SECRET_NAME }}. Access in shell: $VAR or ${{ env.VAR }} in YAML.

### 8. What is actions/checkout and why is it needed?

actions/checkout@v4 checks out the repository code onto the runner. Runners don't automatically have your code. fetch-depth: 0 fetches full history (needed for git-based versioning). sparse-checkout for large repos. persist-credentials: false for security. ref: to checkout specific branch/SHA. For PRs from forks: additional security considerations apply.

### 9. Explain GitHub Actions artifacts.

actions/upload-artifact saves files from a job, downloadable from UI or by subsequent jobs using actions/download-artifact. Use for: passing build outputs between jobs, debugging, providing downloads. Artifacts are stored on GitHub (max 10GB, retention configurable). Larger files: use S3 or other storage. Don't use artifacts for secrets.

### 10. How do you use matrix builds in GitHub Actions?

strategy.matrix allows running a job with different parameter combinations. matrix: {os: [ubuntu, windows, macos], node: [18, 20]}. Creates 6 job instances. ${{ matrix.os }}, ${{ matrix.node }} reference values. include: adds extra combinations, exclude: removes specific combos. fail-fast: false continues even if one matrix job fails. max-parallel controls concurrency.

### 11. What are GitHub Actions reusable workflows?

workflow_call trigger makes a workflow callable by others. Define inputs: and outputs:. Caller: jobs.job_name.uses: org/repo/.github/workflows/deploy.yml@main with: inputs and secrets: inherit. Enables DRY principle across repositories. Reusable workflows run in the context of the calling repository. Nested calls up to 4 levels deep.

### 12. Explain GitHub Actions composite actions.

Composite actions bundle multiple steps into a reusable action (action.yml with runs.using: composite). Can include run steps, uses steps, inputs/outputs. Stored in .github/actions/my-action/ or a separate repo. Used with: uses: ./.github/actions/my-action (local) or org/repo/actions/my-action@v1. Unlike reusable workflows, composite actions run in the caller's job.

### 13. What are GitHub-hosted vs self-hosted runners?

GitHub-hosted: ubuntu-latest, windows-latest, macos-latest — managed by GitHub, fresh per job, 2-4 cores, 7-14GB RAM. Free minutes for public repos, limited for private. Self-hosted: your own infrastructure, custom hardware, persistent, no minutes limit, behind firewall access, required for specific OS/hardware. Use labels to target: runs-on: [self-hosted, linux, gpu].

### 14. How do GitHub Actions secrets work?

Secrets stored in GitHub (Settings > Secrets) — encrypted, masked in logs, never sent to forks. Access: ${{ secrets.NAME }}. Three levels: repository, environment, organization. GITHUB_TOKEN is automatically available with limited permissions. Secrets are NOT available in pull requests from forks (security). Use environment secrets for deployment-specific credentials.

### 15. What is GITHUB_TOKEN and its permissions?

Automatically created per workflow run with short-lived credentials. Default permissions vary by repo settings (permissive vs restrictive). Explicitly set: permissions: contents: read, pull-requests: write, packages: write. Used for: pushing to repo, creating releases, publishing packages, commenting on PRs. tokens expire after workflow. Scope precisely for security (least privilege).

### 16. How do you cache dependencies in GitHub Actions?

actions/cache@v3: key (cache identifier, include hash of lock file), path (directories to cache), restore-keys (fallback keys). Example: key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}. Cache hit restores, miss runs normally and saves after job. Max 10GB per repo, evicted after 7 days unused. Only cache deterministic outputs.

### 17. Explain GitHub Actions environments.

Environments (Settings > Environments) provide: deployment protection rules (required reviewers, wait timer), environment secrets (override repo secrets), environment variables, deployment history. Reference in workflow: environment: production. Required reviewers pause workflow until approved. URL: shows in deployment interface. Branch restrictions limit who can deploy.

### 18. What are GitHub Actions workflow triggers with filters?

push: branches: [main, 'release/**'], paths: ['src/**', '!docs/**']. pull_request: types: [opened, synchronize], branches: [main]. schedule: cron: '0 2 * * *'. workflow_dispatch: inputs. Paths-filter avoids unnecessary runs. branches-ignore/paths-ignore for exclusions. Filter precision reduces costs and noise.

### 19. How do you use continue-on-error and timeout in steps?

continue-on-error: true — step failure doesn't fail the job (useful for optional steps like coverage). timeout-minutes: 5 — kills step after 5 min. Job-level: jobs.job.timeout-minutes: 30. steps.step.outcome: success/failure/cancelled/skipped. Combine: continue-on-error: true then check ${{ steps.step.outcome == 'failure' }}.

### 20. What is workflow_dispatch and how do you add input parameters?

workflow_dispatch allows manual triggering from UI/API. Define inputs: under on.workflow_dispatch.inputs: — type (string/boolean/choice/number/environment), description, required, default, options (for choice). Access: ${{ inputs.environment }}. Trigger via API: POST /repos/:owner/:repo/actions/workflows/:id/dispatches with ref and inputs JSON.

### 21. How do you use concurrency groups to prevent duplicate runs?

concurrency: group: ${{ github.workflow }}-${{ github.ref }} — only one run per workflow+branch. cancel-in-progress: true cancels previous run when new one starts. Useful for: PR checks (cancel stale runs), deployments (queue or cancel). Per-environment: group: deploy-${{ inputs.environment }} prevents parallel deploys to same env.

### 22. What is actions/github-script and when to use it?

github-script allows using GitHub's Octokit.js directly in a workflow step. Use for: commenting on PRs, creating issues, adding labels, posting check annotations, making complex API calls. Example: github.rest.issues.createComment({issue_number: context.issue.number, body: 'Deploy successful'}). Avoids installing GitHub CLI or curl hacks.

### 23. How do you use job outputs between jobs?

Step: echo 'sha=abc123' >> $GITHUB_OUTPUT. Job: jobs.build.outputs: sha: ${{ steps.step-id.outputs.sha }}. Consumer: needs: [build]; env: SHA: ${{ needs.build.outputs.sha }}. Only strings (not complex objects) as outputs. For complex data: use artifacts or external storage. Outputs are only available from successful jobs (unless always() condition).

### 24. Explain GitHub Actions expressions functions.

contains(haystack, needle), startsWith(str, prefix), endsWith(str, suffix), format('{0} {1}', val1, val2), join(array, sep), toJSON(value), fromJSON(string), hashFiles('glob'), success(), failure(), cancelled(), always() — job status functions. Ternary pattern: ${{ condition && 'true-val' || 'false-val' }}. Type coercion: booleans/null follow specific rules.

### 25. What are job services in GitHub Actions?

services: spin up sidecar Docker containers alongside a job. Example: services: postgres: image: postgres:15 env: POSTGRES_PASSWORD: test ports: ['5432:5432'] options: --health-cmd pg_isready --health-retries 5. Access via localhost:5432. Used for: databases, Redis, other dependencies. Available only while job runs. Perfect for integration tests.

## INTERMEDIATE — Security, Deployments & Advanced Patterns

### 26. How do you implement OIDC authentication with cloud providers in GitHub Actions?

OIDC allows passwordless auth to AWS/GCP/Azure. GitHub generates short-lived OIDC token. Configure cloud provider to trust GitHub's OIDC issuer. AWS: aws-actions/configure-aws-credentials@v4 with role-to-assume (no stored secrets!). GCP: google-github-actions/auth. Azure: azure/login with OIDC. Permissions: id-token: write required. More secure than storing long-lived credentials.

### 27. Explain GitHub Actions security hardening.

Least privilege permissions (permissions: block). Pin actions to full commit SHA (not tags). Avoid using pull_request_target with checkout of fork code. Review third-party actions (use Actions Security

Lab scanner). Limit GITHUB_TOKEN scope. Use environment protection rules. Enable Dependabot for action updates. Audit with GitHub's security overview. CODEOWNERS for workflow file protection.

### 28. How do you implement deployment gates in GitHub Actions?

Environment protection rules: required reviewers (manual approval), wait timers (min 0, max 43200 min), branch restrictions. Workflow pauses at environment reference and waits for approval. API: /repos/:owner/:repo/actions/runs/:run_id/pending_deployments to approve programmatically. Custom gates: webhook to external approval system (Jira, ServiceNow) via repository_dispatch.

### 29. How do you build and push Docker images in GitHub Actions?

docker/setup-buildx-action (enables BuildKit). docker/login-action to registry. docker/build-push-action: tags, cache-from/cache-to (registry cache), platforms (multi-arch), build-args, secrets, push: true. docker/metadata-action for automatic tag generation from git refs. Example tags: image:main, image:sha-abc, image:v1.2.3 for releases.

### 30. Explain GitHub Actions for Kubernetes deployments.

Patterns: 1. kubectl apply (azure/k8s-deploy, or raw kubectl). 2. Helm upgrade --install (helm/chart-deployer). 3. Argo CD (trigger sync). 4. Flux (gitops, push manifest changes). Auth: kubeconfig secret, or cloud OIDC + cloud-specific K8s auth (aws-eks-token, gke-gcloud-auth-plugin). Use environments with protection rules for prod. Always verify rollout: kubectl rollout status.

### 31. How do you implement GitOps with GitHub Actions?

Code repo: build image, push to registry, update image tag in manifest repo (via PR or direct push). Manifest repo: Argo CD/Flux watches it, syncs to cluster. Actions: step to update kustomization.yaml or Helm values file with new image SHA. Use separate repos for app code and k8s manifests. Commit bot creates PRs for image updates (safer for production).

### 32. How do you manage multiple environments (dev/staging/prod) with GitHub Actions?

Trigger logic: push to main → deploy dev, PR merge to release → deploy staging, manual dispatch → deploy prod. Use environments for each, with appropriate protection rules. Parameterize with inputs or branch-based conditions. Reusable workflow for deploy logic: .github/workflows/deploy.yml called with environment input. Secrets per environment. Approval gates for production.

### 33. Explain GitHub Actions for infrastructure as code (Terraform).

hashicorp/setup-terraform action. Workflow: terraform init → terraform plan → manual review (pull request comment with plan output) → terraform apply. Use GitHub Actions OIDC for cloud auth (no static credentials). atlantis is an alternative. Store state in remote backend (S3+DynamoDB, Terraform Cloud). Use matrix for multiple environments. Never auto-apply to production without review.

### 34. How do you implement ChatOps with GitHub Actions?

Use repository_dispatch or slash commands via issue comments. actions/github-script to parse /deploy command in PR comment. Trigger workflow_dispatch via API from Slack bot. Post results

back to Slack via slack-notify action or curl to webhook. Create GitHub check runs for deployment status. chatops-deployer patterns: /deploy app to staging, /rollback app.

### 35. Explain GitHub Actions path filtering to optimize CI runs.

dorny/paths-filter action filters based on changed files. Determine which microservices changed, only build/test those. Example: filters: backend: ['backend/**'] frontend: ['frontend/**']. Use outputs in matrix or job conditions. Without filtering: every push rebuilds everything (expensive). path-filter at workflow level (on.push.paths) is coarser; dorny/paths-filter gives per-job control.

### 36. How do you implement parallel test execution in GitHub Actions?

Matrix strategy: split tests by file/suite. Custom sharding: TEST_TOTAL=4 TEST_INDEX=${{ matrix.index }} — each job runs 1/4 of tests. tools like jest --shard, pytest-xdist, playwright --shard. Upload coverage per shard as artifact, merge in final step. Cache test results. jest --testPathPattern can split by regex. Significant speedup for large test suites.

### 37. How do you manage workflow run history and retention?

GitHub retains logs/artifacts 90 days (configurable per repo/org). Set artifact retention: actions/upload-artifact with retention-days: 7. Programmatically delete: gh api /repos/:owner/:repo/actions/runs/:id -X DELETE. Archive important runs. Use GitHub's workflow_run event to trigger cleanup jobs. Branch rules can auto-delete merged branches (reducing run history scope).

### 38. Explain how to create a custom JavaScript action.

index.js: const core = require('@actions/core'); const github = require('@actions/github'); const input = core.getInput('name'); core.setOutput('result', value); core.setFailed('message'). action.yml: name, description, inputs, outputs, runs: using: node20 main: dist/index.js. Use @vercel/ncc to bundle: ncc build index.js -o dist. Commit dist/. Publish to marketplace optionally.

### 39. How do you handle flaky tests in GitHub Actions?

Retry strategy: run tests multiple times, require N consecutive failures before failing job. actions/github-script to rerun failed steps. Custom: run pytest 3 times, exit 0 if any pass. Identify flaky tests: compare test results across runs. Quarantine: skip known flaky tests with tags, track in issue. Fix root cause: async timing issues, shared state, external dependencies. Never ignore; treat as bugs.

### 40. What is GitHub Advanced Security and relevant Actions?

GHAS includes: CodeQL (SAST), Secret Scanning, Dependency Review. codeql-action/init + codeql-action/analyze for code scanning. github/codeql-action/upload-sarif for other SAST tools. dependency-review-action blocks PRs with vulnerable deps. Secret scanning prevents pushing secrets. Results appear in Security tab. Required for compliance (SOC2, ISO27001).

### 41. How do you implement cost optimization for GitHub Actions?

Path filtering (don't run for docs changes). Cache dependencies aggressively. Use self-hosted runners for intensive workloads. Concurrency groups to cancel stale runs. timeout-minutes to prevent

runaway jobs. Reduce matrix jobs. Conditional steps for expensive operations. Spot/preemptible self-hosted runners. Monitor with GitHub's usage reports and billing API.

### 42. Explain GitHub Packages and using it with Actions.

GitHub Packages supports npm, Maven, Gradle, Docker, NuGet, RubyGems. Push Docker image: REGISTRY=ghcr.io, docker push ghcr.io/owner/repo:tag. Auth: GITHUB_TOKEN with packages: write permission. Pull in another repo: packages: read, authenticate in docker/login-action with ${{ secrets.GITHUB_TOKEN }}. Version packages aligned with releases. Private packages require auth even for reads.

### 43. How do you implement release automation with GitHub Actions?

On tag push (v*): build artifacts, run tests, create GitHub release (actions/create-release or gh release create), upload assets (actions/upload-release-asset). Auto-generate release notes from merged PRs (softprops/action-gh-release with generate_release_notes: true). Version bumping: google-github-actions/release-please or semantic-release. Trigger downstream: repository_dispatch.

### 44. How do you implement code quality gates in GitHub Actions?

Required status checks (branch protection) block merges. SonarCloud/SonarQube action with quality gate check. Code coverage threshold: fail if coverage < 80% (lcov, codecov). Lint: eslint, pylint, golangci-lint as required checks. sonarcloud-github-action with sonar.qualitygate.wait=true. Enforce: all checks must pass before merge. PR comments with quality metrics.

### 45. How do you debug GitHub Actions workflow failures?

Enable step debug: secrets.ACTIONS_STEP_DEBUG = true (verbose per-step logging). Runner debug: secrets.ACTIONS_RUNNER_DEBUG = true. Re-run with debug logging from UI. SSH into runner: mxschmitt/action-tmate (opens SSH session, careful with secrets exposure). Add debug steps: run: env (env vars), ls -la (filesystem). Use act CLI for local testing. Check GITHUB_ACTIONS env in local testing.

## ADVANCED — Enterprise, Performance & Troubleshooting

### 46. How do you implement enterprise-grade GitHub Actions at scale?

Org-level reusable workflows in .github repo (auto-available to all repos). Starter workflows. Required status checks enforcement. Organization secrets and variables. Self-hosted runner groups with access policies. Audit log streaming. OIDC provider configuration org-wide. Enterprise: GitHub Connect, GHES (GitHub Enterprise Server) for self-hosted. Policy as code: OPA/Conftest for workflow validation.

### 47. Explain the security risks of pull_request_target.

pull_request_target runs with full repository secrets and WRITE access even for PRs from forks — unlike pull_request (no secrets for forks). NEVER checkout and execute fork code with

pull_request_target (code injection!). Use for: updating PR status, adding labels without code execution. If you need both secrets and fork code: manual approval step first, then run in separate PR-approved workflow.

### 48. How do you implement dynamic matrix from API or file?

Step outputs JSON: echo 'matrix={"include":[{"service":"api"},{"service":"worker"}]}' >> $GITHUB_OUTPUT. Job uses: strategy.matrix: ${{ fromJSON(needs.prep.outputs.matrix) }}. Or read from file: matrix=$(cat services.json); echo "matrix=$matrix" >> $GITHUB_OUTPUT. Dynamic matrices enable: building only changed services, parameterizing from external config, scaling tests based on size.

### 49. How do you implement workflow templating across many repositories?

Org-wide starter workflows in .github/workflow-templates/. Reusable workflows called from template. Cookiecutter/Copier for initial setup. GitHub's repository templates. Actions from internal registry (GHES or public). Centralized self-hosted runner management. .github repo for org-level defaults. GitHub App to auto-create/update workflow files across repos (using octokit/rest API). Policy enforcement via repo ruleset.

### 50. Explain GitHub Actions runner scaling strategies.

Self-hosted: manual scaling (VMs), auto-scaling via Terraform with webhook triggers, actions-runner-controller (ARC) on Kubernetes (scales runner pods on demand). AWS CodeBuild runners. Custom: webhook → Lambda → EC2 spot instance. ARC supports: ephemeral runners (fresh per job), runner scale sets, spot instances. GitHub-hosted larger runners: 4/8/16/32-core, 32-64GB RAM (additional cost).

### 51. How do you implement policy enforcement on GitHub Actions workflows?

Required status checks in branch protection. CODEOWNERS for .github/workflows/ requiring review from security team. OPA/Conftest: validate workflow YAML against policies in CI. GitHub Actions policy enforcement (enterprise): control which actions are allowed (from marketplace vs internal). Dependabot for action version updates. Audit log alerts for workflow modifications.

### 52. Debug: A workflow runs on push but not on pull_request. How do you investigate?

Check: 1. Branch protection rules excluding PR event. 2. on: triggers — is pull_request listed? types correct? 3. paths filter excluding modified files. 4. Branch filter not matching PR base/head. 5. Workflow file only exists on main branch (PRs run workflow from base branch, not PR branch for security). 6. Actor restrictions or repo settings. Check Actions tab → filter by event.

### 53. How do you implement zero-downtime deployment workflow?

1. Build and push image (tag: sha). 2. Deploy to staging: kubectl set image, verify rollout status. 3. Run smoke tests. 4. Deployment gate (manual approval for prod). 5. Deploy to prod: Helm upgrade --atomic (auto-rollback on failure), or Argo CD sync. 6. Health check: wget/curl prod endpoint. 7. Automated rollback step: if health check fails: helm rollback. 8. Notify (Slack/PagerDuty).

### 54. How do you pass secrets securely between jobs?

Secrets cannot be passed via outputs (outputs are visible in logs). Options: 1. Use organization/environment secrets (available to all jobs). 2. Intermediate: generate and store short-lived token in secret manager, pass secret manager path as output. 3. Encrypt with GPG: encrypt in job1, decrypt in job2 with shared key. 4. Write to artifact (only if encrypted, artifacts are downloadable). Best: use shared secret manager.

### 55. Explain GitHub Actions cache eviction and strategy for monorepo.

Cache key must be unique and include content hash: runner.os + hash of relevant lock files. Restore-keys for fallback: less specific keys. Cache evicted after 7 days unused or when 10GB repo limit exceeded. Monorepo: per-package cache: runner.os-node-${{ hashFiles('packages/api/package-lock.json') }}. Separate caches for each package/service. actions/cache@v3 has better cache hit reporting.

## REAL-WORLD SCENARIOS

### 56. Design a complete CI/CD workflow for a microservices platform.

Workflow: 1. Trigger on push/PR. 2. Detect changed services (dorny/paths-filter). 3. Matrix build: each changed service → test + docker build + trivy scan. 4. Push to ECR with commit SHA tag. 5. Update Helm values in manifest repo (separate job). 6. Staging auto-deploy (Argo CD sync). 7. E2E tests against staging. 8. Production deploy gate (required reviewers). 9. Prod deploy. 10. Slack notification with deploy summary.

### 57. How would you implement a workflow that handles infrastructure and application deployment?

Separate jobs: infra (Terraform apply) and app (Helm upgrade) with needs: [infra]. Terraform outputs (cluster endpoint) passed to Helm job via outputs. OIDC auth to AWS/GCP for both. Terraform plan posted as PR comment for review. Auto-apply to dev, manual gate for prod. State locking (DynamoDB) prevents concurrent applies. Drift detection: scheduled workflow comparing actual vs desired.

### 58. Design a security scanning pipeline for all repositories in an org.

Org-level reusable workflow. Components: 1. CodeQL SAST (codeql-action). 2. Dependency review (dependency-review-action). 3. Container scan (trivy). 4. Secret detection (gitleaks/truffleHog). 5. IaC scan (checkov for Terraform/K8s). Results: SARIF uploaded to GitHub Security. Critical findings fail PR, high/medium as warnings. Weekly full scans. Aggregated dashboard via GitHub API → Grafana.

### 59. How would you implement a multi-cloud deployment workflow?

Matrix or sequential jobs for AWS/GCP/Azure. OIDC tokens per cloud provider (separate id-token setup). Cloud-specific CLI setup in each job. Deploy same Helm chart with per-cloud values files. Test each deployment independently. Rollback per-cloud. DNS: Route53/Cloud DNS/Azure DNS updated after successful deploy. Monitor: unified alerting regardless of cloud. Terraform modules per cloud with shared state backend.

### 60. How do you handle a GitHub Actions workflow that's been compromised?

Immediate: rotate all secrets used in that workflow. Audit log: who modified the workflow file and when. Review changed workflow: check for exfiltration (curl to external, base64 encoded), persistence (creating new secrets, webhooks), lateral movement (cross-repo access). Revoke GITHUB_TOKEN (it expires anyway). Revoke OIDC role assumptions. Notify security team. CODEOWNERS protection on .github/workflows/. Re-audit all org secrets.